

```
In [1]: !pip install torch torchvision torchaudio scikit-learn medmnist
```

```
Requirement already satisfied: torch in /opt/conda/lib/python3.13/site-packages (2.13.0)
Requirement already satisfied: torchvision in /opt/conda/lib/python3.13/site-packages (0.28.0)
Requirement already satisfied: torchaudio in /opt/conda/lib/python3.13/site-packages (2.11.0)
Requirement already satisfied: scikit-learn in /opt/conda/lib/python3.13/site-packages (1.9.0)
Requirement already satisfied: medmnist in /opt/conda/lib/python3.13/site-packages (3.0.2)
Requirement already satisfied: filelock in /opt/conda/lib/python3.13/site-packages (from torch) (3.32.2)
Requirement already satisfied: typing-extensions>=4.10.0 in /opt/conda/lib/python3.13/site-packages (from torch) (4.16.0)
Requirement already satisfied: setuptools>=77.0.3 in /opt/conda/lib/python3.13/site-packages (from torch) (82.0.1)
Requirement already satisfied: sympy>=1.13.3 in /opt/conda/lib/python3.13/site-packages (from torch) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in /opt/conda/lib/python3.13/site-packages (from torch) (3.6.1)
Requirement already satisfied: jinja2 in /opt/conda/lib/python3.13/site-packages (from torch) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in /opt/conda/lib/python3.13/site-packages (from torch) (2026.7.0)
Requirement already satisfied: cuda-toolkit==13.0.3 in /opt/conda/lib/python3.13/site-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparse,nvjitlink,nvrtc,nvtx]==13.0.3; platform_system == "Linux"->torch) (13.0.3.0)
Requirement already satisfied: cuda-bindings<14,>=13.0.3 in /opt/conda/lib/python3.13/site-packages (from torch) (13.3.1)
Requirement already satisfied: nvidia-cudnn-cu13==9.20.0.48 in /opt/conda/lib/python3.13/site-packages (from torch) (9.20.0.48)
Requirement already satisfied: nvidia-cusparselt-cu13==0.8.1 in /opt/conda/lib/python3.13/site-packages (from torch) (0.8.1)
Requirement already satisfied: nvidia-nccl-cu13==2.29.7 in /opt/conda/lib/python3.13/site-packages (from torch) (2.29.7)
Requirement already satisfied: nvidia-nvshmem-cu13==3.4.5 in /opt/conda/lib/python3.13/site-packages (from torch) (3.4.5)
Requirement already satisfied: triton==3.7.1 in /opt/conda/lib/python3.13/site-packages (from torch) (3.7.1)
```

Requirement already satisfied: nvidia-cublas==13.1.1.3.* in /opt/conda/lib/python3.13/site-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparsenvjitlink,nvrtc,nvtx]==13.0.3; platform_system == "Linux"->torch) (13.1.1.3)

Requirement already satisfied: nvidia-cuda-nvrtc==13.0.88.* in /opt/conda/lib/python3.13/site-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparsenvjitlink,nvrtc,nvtx]==13.0.3; platform_system == "Linux"->torch) (13.0.88)

Requirement already satisfied: nvidia-cuda-runtime==13.0.96.* in /opt/conda/lib/python3.13/site-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparsenvjitlink,nvrtc,nvtx]==13.0.3; platform_system == "Linux"->torch) (13.0.96)

Requirement already satisfied: nvidia-cufft==12.0.0.61.* in /opt/conda/lib/python3.13/site-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparsenvjitlink,nvrtc,nvtx]==13.0.3; platform_system == "Linux"->torch) (12.0.0.61)

Requirement already satisfied: nvidia-nvjitlink<14,>=13.0.88 in /opt/conda/lib/python3.13/site-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparsenvjitlink,nvrtc,nvtx]==13.0.3; platform_system == "Linux"->torch) (13.3.33)

Requirement already satisfied: nvidia-cufile==1.15.1.6.* in /opt/conda/lib/python3.13/site-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparsenvjitlink,nvrtc,nvtx]==13.0.3; platform_system == "Linux"->torch) (1.15.1.6)

Requirement already satisfied: nvidia-cuda-cupti==13.0.85.* in /opt/conda/lib/python3.13/site-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparsenvjitlink,nvrtc,nvtx]==13.0.3; platform_system == "Linux"->torch) (13.0.85)

Requirement already satisfied: nvidia-curand==10.4.0.35.* in /opt/conda/lib/python3.13/site-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparsenvjitlink,nvrtc,nvtx]==13.0.3; platform_system == "Linux"->torch) (10.4.0.35)

Requirement already satisfied: nvidia-cusolver==12.0.4.66.* in /opt/conda/lib/python3.13/site-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparsenvjitlink,nvrtc,nvtx]==13.0.3; platform_system == "Linux"->torch) (12.0.4.66)

Requirement already satisfied: nvidia-cusparsenvjitlink,nvrtc,nvtx]==13.0.3; platform_system == "Linux"->torch) (12.6.3.3)

Requirement already satisfied: nvidia-nvtx==13.0.85.* in /opt/conda/lib/python3.13/site-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparsenvjitlink,nvrtc,nvtx]==13.0.3; platform_system == "Linux"->torch) (13.0.85)

Requirement already satisfied: cuda-pathfinder>=1.4.2 in /opt/conda/lib/python3.13/site-packages (from cuda-bindings<14,>=13.0.3->torch) (1.6.0)

Requirement already satisfied: numpy in /opt/conda/lib/python3.13/site-packages (from torchvision) (2.5.1)

Requirement already satisfied: pillow!>=8.3.*,>=5.3.0 in /opt/conda/lib/python3.13/site-packages (from torchvision) (12.3.0)

Requirement already satisfied: scipy>=1.10.0 in /opt/conda/lib/python3.13/site-packages (from scikit-learn) (1.18.0)

Requirement already satisfied: joblib>=1.4.0 in /opt/conda/lib/python3.13/site-packages (from scikit-learn) (1.5.3)

Requirement already satisfied: narwhals>=2.0.1 in /opt/conda/lib/python3.13/site-packages (from scikit-learn) (2.24.0)

Requirement already satisfied: threadpoolctl>=3.5.0 in /opt/conda/lib/python3.13/site-packages (from scikit-learn) (3.6.0)

Requirement already satisfied: pandas in /opt/conda/lib/python3.13/site-packages (from medmnist) (3.0.5)

Requirement already satisfied: scikit-image in /opt/conda/lib/python3.13/site-packages (from medmnist) (0.26.0)

Requirement already satisfied: tqdm in /opt/conda/lib/python3.13/site-packages (from medmnist) (4.67.3)

Requirement already satisfied: fire in /opt/conda/lib/python3.13/site-packages (from medmnist) (0.7.1)

Requirement already satisfied: mpmath<1.4,>=1.1.0 in /opt/conda/lib/python3.13/site-packages (from sympy>=1.13.3->torch) (1.3.0)

Requirement already satisfied: termcolor in /opt/conda/lib/python3.13/site-packages (from fire->medmnist) (3.3.0)

Requirement already satisfied: MarkupSafe>=2.0 in /opt/conda/lib/python3.13/site-packages (from jinja2->torch) (3.0.3)

Requirement already satisfied: python-dateutil>=2.8.2 in /opt/conda/lib/python3.13/site-packages (from pandas->medmnist) (2.9.0.post0)

Requirement already satisfied: six>=1.5 in /opt/conda/lib/python3.13/site-packages (from python-dateutil>=2.8.2->pandas->medmnist) (1.17.0)

Requirement already satisfied: imageio!=2.35.0,>=2.33 in /opt/conda/lib/python3.13/site-packages (from scikit-image->medmnist) (2.37.4)

Requirement already satisfied: tifffile>=2022.8.12 in /opt/conda/lib/python3.13/site-packages (from scikit-image->medmnist) (2026.7.31)

Requirement already satisfied: packaging>=21 in /opt/conda/lib/python3.13/site-packages (from scikit-image->medmnist) (26.2)

Requirement already satisfied: lazy-loader>=0.4 in /opt/conda/lib/python3.13/site-packages (from scikit-image->medmnist) (0.5)

In [2]:

```

"""
Created on Mon Aug4 17:35:36 2025

@author: vigo
"""

import os
import sys
import argparse
import time
import pickle
from tqdm import trange
import numpy as np
import PIL
import copy
import random
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
import torch.utils.data as data
import torchvision
import torchvision.transforms as transforms
from torchvision.models import resnet18, resnet50
import matplotlib.pyplot as plt
from scipy.io import savemat

#Related medmnist dataset
import medmnist
from medmnist import INFO, Evaluator

# Check for Cuda
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

```

```

seed = 27
torch.manual_seed(seed)
random.seed(seed)

# Hyper-parameters
n_epochs = 10
batch_size = 128
lr = 0.0005

#Network Hyper-parameters
hidden_size = 512

data_flag = 'organamnist'
info = INFO[data_flag]
task = info['task']
n_channels = info['n_channels']
n_classes = len(info['label'])
DataClass = getattr(medmnist, info['python_class'])

#To work with 28x28 size images
#data_transform = transforms.Compose([
    #transforms.ToTensor(),
    #transforms.Normalize(mean=[.5], std=[.5])])

### To work with 224x224 resolution images
data_transform = transforms.Compose(
    [transforms.Resize((224, 224), interpolation=PIL.Image.NEAREST),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.5], std=[0.5])])

# Image Size 28x28. Set RGB =True if the dataset contains colour images
train_dataset = DataClass(split='train', transform=data_transform, download=True)
val_dataset = DataClass(split='val', transform=data_transform, download=True, as
test_dataset = DataClass(split='test', transform=data_transform, download=True,

# Loaders that give 64 example batches
train_loader = data.DataLoader(dataset=train_dataset,
                                batch_size=batch_size,
                                shuffle=True)

val_loader = data.DataLoader(dataset=val_dataset,
                              batch_size=batch_size,
                              shuffle=False)

test_loader = data.DataLoader(dataset=test_dataset,
                              batch_size=batch_size,
                              shuffle=False)

image, label = train_dataset[0] # Get one sample image
print(f"Image shape: {image.shape}")

```

Image shape: torch.Size([1, 224, 224])

```

In [3]: # select 16 random indices for plotting
indices = random.sample(range(len(train_dataset)), 16)

```

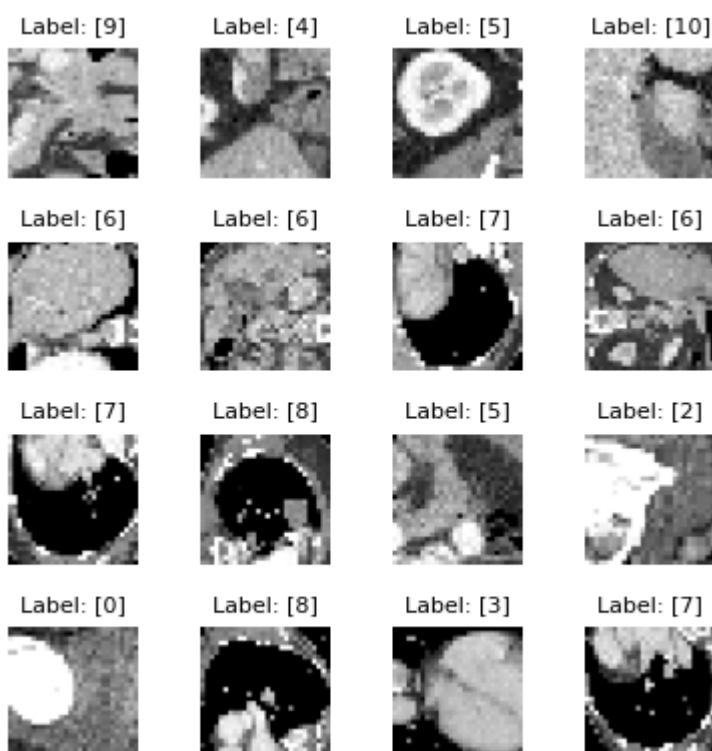
```
plt.figure(figsize=(4, 4))

for i, idx in enumerate(indices):
    image, label = train_dataset[idx]

    # If tensor, convert to 2D array (grayscale)
    if hasattr(image, 'squeeze'):
        image = image.squeeze() # remove channel dimension if present

    plt.subplot(4, 4, i + 1)
    plt.imshow(image, cmap='gray')
    plt.title(f"Label: {label}", fontsize=8)
    plt.axis('off')

plt.tight_layout()
plt.show()
```



```
In [4]: class AlexNet(nn.Module):
    def __init__(self, in_channels, num_classes):
        super(AlexNet, self).__init__()
        self.conv1 = nn.Conv2d(in_channels=in_channels, out_channels=96, kernel_
        self.pool1 = nn.MaxPool2d(kernel_size=3, stride=2)
        self.conv2 = nn.Conv2d(in_channels=96, out_channels=256, kernel_size=5,
        self.pool2 = nn.MaxPool2d(kernel_size=3, stride=2)
        self.conv3 = nn.Conv2d(in_channels=256, out_channels=384, kernel_size=3,
        self.conv4 = nn.Conv2d(in_channels=384, out_channels=384, kernel_size=3,
        self.conv5 = nn.Conv2d(in_channels=384, out_channels=256, kernel_size=3,
        self.pool3 = nn.MaxPool2d(kernel_size=3, stride=2)
        self.fc1 = nn.Linear(5 * 5 * 256, 4096)
        self.d1= nn.Dropout(p=0.5)
        self.fc2 = nn.Linear(4096, 1024)
        self.d2 =nn.Dropout(p=0.5)
        self.fc3 = nn.Linear(1024, num_classes)

    def forward(self, x):
        # -> n, 3, 32, 32
```

```

        x = self.pool1(F.relu(self.conv1(x)))
        x = self.pool2(F.relu(self.conv2(x)))
        x = F.relu(self.conv3(x))
        x = F.relu(self.conv4(x))
        x = self.pool3(F.relu(self.conv5(x)))
        x = x.view(-1, 5 * 5 * 256)
        x = self.d1(F.relu(self.fc1(x)))
        #x = F.relu(self.bn1(self.fc1(x)))
        x = self.d2(F.relu(self.fc2(x)))
        x = self.fc3(x)
        #return torch.log_softmax(x, dim=1)      #for negative LL loss
        return x

conv_model = AlexNet(in_channels=n_channels, num_classes=n_classes).to(device)
# Loss and optimizer
criterion = nn.CrossEntropyLoss()    #One can try NLL loss
optimizer = torch.optim.Adam(conv_model.parameters(), lr=lr, betas=(0.9, 0.999))

```

```

In [5]: # Training
def train(model, dataloader, criterion, optimizer, device):
    model.train()
    train_correct = 0
    total_samples = 0

    for inputs, targets in dataloader:
        inputs, targets = inputs.to(device), torch.squeeze(targets, 1).long().to(device)

        optimizer.zero_grad()
        outputs = model(inputs)
        loss = criterion(outputs, targets)
        loss.backward()
        optimizer.step()

        _, predictions = torch.max(outputs, 1)
        train_correct += (predictions == targets).sum().item()
        total_samples += targets.size(0)

    train_acc = (train_correct / total_samples) * 100
    return train_acc, loss.item()

# Validation/Test
def val(model, dataloader, criterion, device):
    model.eval()
    val_correct = 0
    total_samples = 0
    val_loss = 0.0

    with torch.no_grad():
        for inputs, targets in dataloader:
            inputs, targets = inputs.to(device), torch.squeeze(targets, 1).long().to(device)

            outputs = model(inputs)
            loss = criterion(outputs, targets)
            _, predictions = torch.max(outputs, 1)

            val_correct += (predictions == targets).sum().item()
            total_samples += targets.size(0)
            val_loss += loss.item()

```

```

val_acc = (val_correct / total_samples) * 100
avg_val_loss = val_loss / len(dataloader)
return val_acc, avg_val_loss

```

```

In [6]: Train_Acc = []
        Train_Loss = []

        Val_Acc = []
        Val_Loss = []

        for epoch in range(n_epochs):
            train_acc, train_loss = train(conv_model, train_loader, criterion, optimizer)
            val_acc, val_loss = val(conv_model, val_loader, criterion, device)
            print (f'Epoch--[{epoch+1}], Train_Accuracy: {train_acc}, Validation_Accuracy: {val_acc}')
            Train_Acc.append(train_acc)
            Train_Loss.append(train_loss)
            Val_Acc.append(val_acc)
            Val_Loss.append(val_loss)

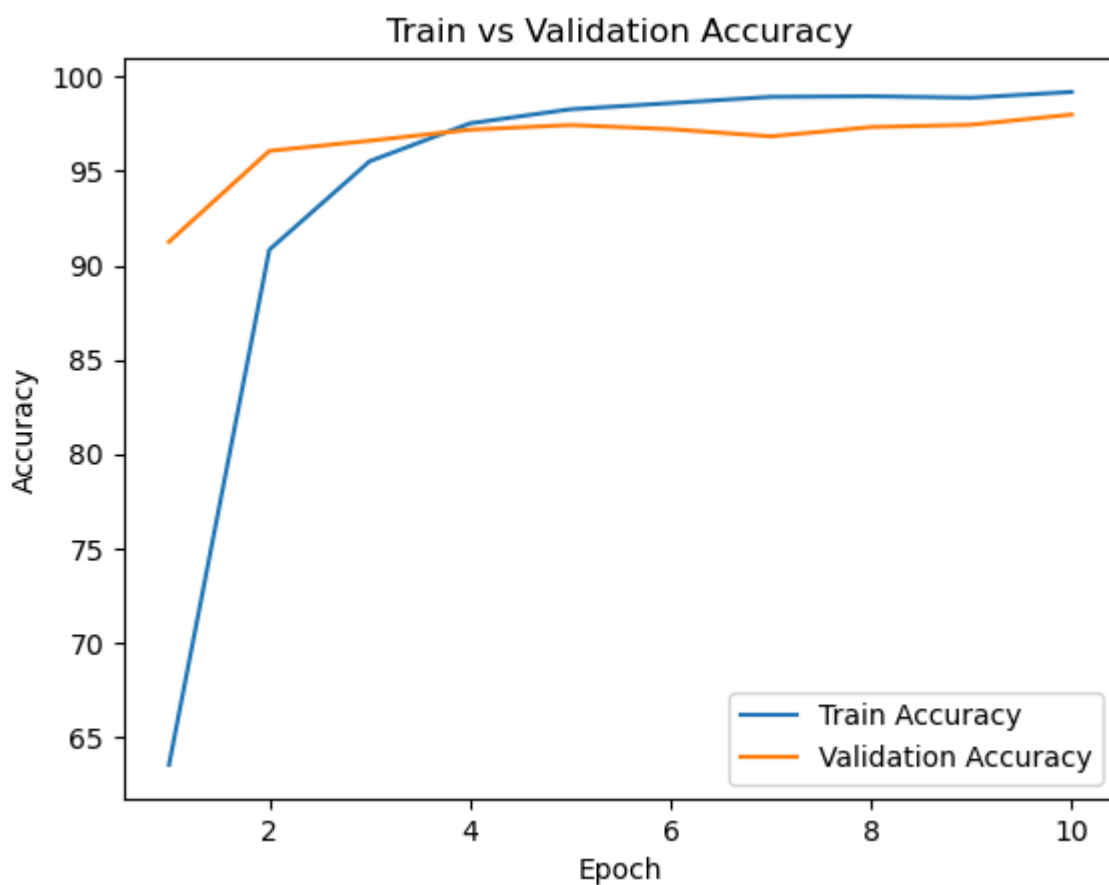
        # Plot Epoch vs Accuracy
        plt.figure()
        plt.plot(range(1, n_epochs+1), Train_Acc, label='Train Accuracy')
        plt.plot(range(1, n_epochs+1), Val_Acc, label='Validation Accuracy')
        plt.xlabel('Epoch')
        plt.ylabel('Accuracy')
        plt.title('Train vs Validation Accuracy')
        plt.legend()
        #plt.grid(True)
        plt.show()

        # Plot Epoch vs Loss
        plt.figure()
        plt.plot(range(1, n_epochs+1), Train_Loss, label='Train Loss')
        plt.plot(range(1, n_epochs+1), Val_Loss, label='Validation Loss')
        plt.xlabel('Epoch')
        plt.ylabel('Loss')
        plt.title('Train vs Validation Loss')
        plt.legend()
        #plt.grid(True)
        plt.show()

        #Testing
        test_acc, test_loss = val(conv_model, test_loader, criterion, device)
        print (f'Test_Accuracy: {test_acc}')

```

```
Epoch--[1], Train_Accuracy: 63.51957408639797, Validation_Accuracy: 91.2494222769
9893
Epoch--[2], Train_Accuracy: 90.8307051300599, Validation_Accuracy: 96.07148359266
677
Epoch--[3], Train_Accuracy: 95.52096293510026, Validation_Accuracy: 96.6106917270
0662
Epoch--[4], Train_Accuracy: 97.53190011863083, Validation_Accuracy: 97.1961177014
3275
Epoch--[5], Train_Accuracy: 98.27840629611411, Validation_Accuracy: 97.4426128485
5953
Epoch--[6], Train_Accuracy: 98.60536442811261, Validation_Accuracy: 97.2269295948
236
Epoch--[7], Train_Accuracy: 98.938109429704, Validation_Accuracy: 96.841780927438
Epoch--[8], Train_Accuracy: 98.96993721246491, Validation_Accuracy: 97.3347712216
9157
Epoch--[9], Train_Accuracy: 98.89181447296085, Validation_Accuracy: 97.4580187952
5496
Epoch--[10], Train_Accuracy: 99.19851856138422, Validation_Accuracy: 97.997226929
59483
```





Test_Accuracy: 88.29452131848352

In []:

In []: